

TD1 – Pointeurs, Récursivité, Complexité et Structures de Données

Ce premier Travaux Dirigés porte sur les notions suivantes :

- Compréhension et manipulation des **pointeurs**
- Maîtrise de la **récursivité**
- Calcul de la **complexité temporelle** (nombre d'opérations)
- Structures linéaires : **listes chaînées, piles, files, listes doublement chaînées, listes circulaires**

Exercice 1 : Analyse d'un programme récursif

Considérons le programme suivant :

```
void m(int n){  
  
    if(n == 0) return;  
  
    m(n - 1);  
  
    cout << n << endl;  
  
}  
  
int main(){  
  
    m(5);  
  
    return 0;  
  
}
```

1. Donner l'**affichage exact** produit par ce programme.
2. **Justifier** le résultat obtenu (comportement récursif, pile d'appels...).

Exercice 2 : Analyse de complexité

Soit l'algorithme suivant :

```
pour i ← 1 à n faire  
  
    pour j ← 1 à i faire  
  
        afficher(j)
```

fin pour

fin pour

1. Combien de fois l'opération d'initialisation de la variable j est-elle exécutée ?
2. Combien de fois l'opération d'incrément de j est-elle exécutée ?

Exercice 3 : Structures, tableaux et pointeurs

On considère la structure suivante qui permet de représenter un étudiant :

```
struct Etudiant {  
    int matricule;  
    float moyenne;  
    char nom[30];  
};
```

On dispose également du tableau suivant, contenant **n étudiants** :

```
Etudiant T[100];  
int n;
```

3.1 Saisir les données des n étudiants dans le tableau T en utilisant uniquement des pointeurs.

3.2 Écrire une fonction qui recherche, à l'aide d'un pointeur, l'étudiant ayant la meilleure moyenne.

3.3 Écrire une fonction qui échange deux étudiants en utilisant des pointeurs.

Prototype conseillé :

```
void echanger(Etudiant* a, Etudiant* b);
```

3.4 Trier le tableau T par ordre décroissant des moyennes en utilisant la fonction d'échange et uniquement des pointeurs.

3.5 Afficher le contenu du tableau trié en utilisant un pointeur de parcours.

Exercice 4 : Listes chaînées et piles

Écrire les algorithmes permettant de :

1. Trouver l'élément situé au milieu d'une liste chaînée simple.
2. Inverser les éléments d'une pile, éventuellement en utilisant une seconde structure.
3. Diviser une pile P1 d'entiers en deux piles :

- P2 contient les entiers **impairs**
- P3 contient les entiers **pairs**

À la fin, **P1 doit être vide**.

Exercice 5 : Implémentation d'une pile avec un tableau

1. Proposer une structure de données permettant de représenter une pile (tableau + sommet).
2. Écrire les algorithmes suivants :
 - pileVide
 - empiler
 - Depiler
 - Sommet

Exercice 6 : Implémentation d'une file avec une liste chaînée

À partir de la structure vue en cours, écrire les fonctions C++ réalisant :

1. enfiler (insertion en queue)
2. defiler (retrait en tête)
3. premier (consultation de l'élément en tête)

Exercice 7 : Listes avancées — LDC, LC, LDCC

Rappels :

- LDC (liste doublement chaînée) : chaque élément possède un pointeur vers le suivant et le précédent.
- LC (liste circulaire) : le dernier élément pointe vers le premier.
- LDCC (liste doublement chaînée circulaire) : combinaison des deux, double chaînage et circularité.

Travail demandé :

1. Modifier la structure de liste simple pour permettre de manipuler :
 - une LDC
 - une LC
 - une LDCC
2. Écrire une fonction d'insertion **en tête** pour :
 - une LDC
 - une LC
 - une LDCC

Exercice 8 : Implémentation d'une file avec un tableau

Proposer une structure de données permettant de gérer une file à l'aide d'un tableau (indices début/fin, circulaire).

Implémenter les opérations principales :

- fileVide
- enfiler
- defiler
- premier